Trigonometry-Free TSP Heuristics for Embedded Systems: A Compass-Based Matching Approach

Dogan Balban

Independent Research — March 2026

Abstract

Classical TSP heuristics require square-root and trigonometric evaluations that are prohibitively expensive on hardware without floating-point units (FPUs)—microcontrollers, FPGAs, and sensor nodes. We present *Directional Sparse Matching* (DSM), a TSP heuristic that uses **exclusively integer arithmetic**: addition, subtraction, multiplication by two fixed constants, and comparison. No $\sqrt{\cdot}$, arctan, sin, or cos is evaluated at any stage. The method represents each city by its k nearest neighbors' squared distances and discrete 16-point compass directions, then discovers neighbor pairs via an *antiparallel matching rule* filtered by global reference points ("lighthouses"). We are transparent about costs: the initial k-NN computation remains $O(N^2)$ comparisons—identical to Nearest Neighbor—but each comparison costs 5 integer cycles versus 75 cycles (with software $\sqrt{\cdot}$) on a typical 8-bit MCU, yielding a 4–5× wall-clock speedup. Tour quality is 1–7% below Nearest Neighbor, a deliberate trade-off for arithmetic minimalism. On constrained hardware where no FPU exists, DSM provides a TSP heuristic where previously none was practical.

Keywords: TSP, integer arithmetic, embedded systems, FPGA, microcontroller, compass direction, trigonometry-free

1 Introduction

The Travelling Salesman Problem asks for the shortest Hamiltonian cycle through N cities [1]. Decades of research have produced powerful heuristics: Nearest Neighbor (NN), the Christofides–Serdyukov 3/2-approximation [3], and Lin–Kernighan local search [9]. These methods assume access to Euclidean distances, each requiring a $\sqrt{\cdot}$ evaluation.

On general-purpose CPUs with hardware FPUs, $\sqrt{\cdot}$ costs a few nanoseconds. But a growing class of routing problems runs on hardware *without* FPUs:

- **8/16-bit microcontrollers** (ATmega328, MSP430, PIC): dominant in IoT sensor networks. Software $\sqrt{\cdot}$ via Newton iteration: 50–100 cycles per call.
- **FPGAs and ASICs**: used in logistics and industrial routing. Transcendental functions require dedicated IP cores or lookup tables.
- **Low-power drones and robots**: battery-constrained; every CPU cycle costs energy.

On an ATmega328 running at 16 MHz, computing a single $\sqrt{\cdot}$ takes $\sim$5 μs. For N=100 cities, Nearest Neighbor requires $\binom{100}{2}$=4,950 such evaluations—25 ms of pure square-root computation. For N=1,000: 2.5 seconds. Adding arctan for angular methods (Sweep [6]) multiplies this by 2–3×.

This paper asks: Can we build a TSP heuristic using *only* integer addition, subtraction, multiplication, and comparison? What is the cost in tour quality?

1.1 Contribution and Honest Scope

We present *Directional Sparse Matching* (DSM) and state clearly what it is and is not:

What DSM is:

- A complete TSP heuristic pipeline using zero transcendental or square-root functions.
- A 4–5× speedup over NN on integer-only hardware (same comparison count, cheaper per-comparison cost).
- A compact $10N$-value representation of the city graph after initial computation, enabling efficient storage and transmission over narrowband channels.

- A novel neighbor-discovery method (antiparallel matching with lighthouse filtering) that is, to our knowledge, not described in prior TSP literature.

What DSM is not:

- It is *not* faster than NN on modern CPUs with hardware FPUs.
- It does *not* produce better tours than NN (tours are 1–7% longer).
- It does *not* reduce the number of comparisons in Phase 1 (k-NN search remains $O(N^2)$).
- It provides *no* worst-case approximation guarantee.

The value proposition is narrow but real: **on hardware without FPUs, DSM makes TSP heuristics practical where they previously were not.**

2 Method

2.1 Compass16: Direction Without Trigonometry

We classify displacement vectors $(\Delta x, \Delta y)$ into 16 compass sectors using only threshold comparisons.

Definition 1 (Compass16). *Let* $a_x{=}|\Delta x|$, $a_y{=}|\Delta y|$, *and define* $\tau_1{=}\tan(11.25°){\approx}0.199$, $\tau_2{=}\tan(33.75°){\approx}0.668$ *as precomputed constants. The direction* $\theta \in \Theta_{16}$ *is determined by comparing* a_x *against* $a_y{\cdot}\tau_1$ *and* $a_y{\cdot}\tau_2$ *(or symmetrically for the horizontal case), plus sign checks on* Δx *and* Δy.

Proposition 1 (Cost). *Each Compass16 evaluation requires exactly: 2 subtractions, 2 absolute values, 2 multiplications by fixed constants (representable as* 51/256 *and* 171/256 *for shift-multiply on 8-bit hardware), and 2–3 comparisons. Total:* ∼12 *integer cycles.*

For comparison: arctan via CORDIC requires ∼80–200 cycles; $\sqrt{\cdot}$ via Newton iteration requires ∼50–100 cycles on the same hardware [2].

2.2 Squared Distance

Euclidean distance $d = \sqrt{(\Delta x)^2 + (\Delta y)^2}$ requires $\sqrt{\cdot}$. We use $d^2 = (\Delta x)^2 + (\Delta y)^2$ throughout.

Lemma 1. $d^2(A,B) < d^2(A,C) \Leftrightarrow d(A,B) < d(A,C)$, *since* $\sqrt{\cdot}$ *is monotone. The* k *nearest neighbors by* d^2 *are identical to those by* d.

Cost per d^2: 2 subtractions + 2 multiplications + 1 addition = 5 integer cycles. Cost per d (for NN): the same 5 cycles + 50–100 cycles for $\sqrt{\cdot}$ = 55–105 cycles.

2.3 Door Model

Each city stores its k=3 nearest neighbors, each with d^2 and Compass16 direction. Additionally, each city stores its direction and d^2 to m=2 lighthouse reference points (fixed coordinates). Total per city: $2k + 2m = 10$ values.

2.4 Antiparallel Matching

Theorem 1. *If city A has a door pointing to city B with direction* θ *and squared distance* d^2, *then B has a door pointing to A with direction* $\overline{\theta}$ *(the antiparallel: N↔S, NE↔SW, etc.) and the same* d^2.

Matching rule: Two doors from different cities match if (1) their d^2 values agree within relative tolerance ϵ=2% and (2) their directions are antiparallel.

Lighthouse filter: A match is accepted only if both cities see each lighthouse from similar directions ($\leq$ 2 compass steps apart) and similar d^2 ($\leq$ 4% of map scale squared). This suppresses false positives without additional trigonometry.

2.5 Tour Construction

Confirmed matches form edges in a sparse graph (max degree 2 per city). This yields chains (simple paths), which are merged by connecting closest endpoints, then improved by 2-opt [4].

3 Honest Complexity Analysis

We compare DSM and NN operation-by-operation. Both have the same asymptotic comparison count; the difference is cost per comparison.

Table 1: Per-edge cost on an 8-bit MCU without FPU (e.g., ATmega328 at 16 MHz). DSM and NN perform the same $O(N^2/2)$ pairwise comparisons.

Operation	DSM	NN
d^2 computation	5 cyc	5 cyc
$\sqrt{d^2}$ (Newton, 8 iter.)	—	70 cyc
Compass16	12 cyc	—
Total per edge	**17 cyc**	**75 cyc**
Speedup	**4.4×**	

Critical clarification. DSM does *not* reduce the number of comparisons. Phase 1 (k-NN search) requires $N(N-1)/2$ pairwise d^2 evaluations—identical to NN. The speedup comes exclusively from eliminating $\sqrt{\cdot}$ per comparison. On a CPU with a hardware FPU where $\sqrt{\cdot}$ costs ~1 cycle, there is no advantage.

Table 2: Total wall-clock estimate on ATmega328 (16 MHz). Both methods: $N(N-1)/2$ comparisons.

N	Comparisons	DSM time	NN time
100	4,950	5.3 ms	23.4 ms
300	44,850	47.7 ms	210.0 ms
1,000	499,500	531 ms	2.34 s

Post-Phase 1 data. After the $O(N^2)$ initialization, the city graph is compressed to $10N$ values. This compact representation enables:

- *Transmission*: 10,000 cities → 100 KB vs. 50 MB (full matrix). Fits in a LoRa or BLE packet sequence.
- *Incremental updates*: Adding one city requires N new d^2 comparisons (to find its 3 nearest neighbors), not rebuilding the entire matrix.
- *Caching*: The $10N$ representation can be stored in EEPROM for repeated route planning over the same city set with different constraints.

4 Benchmarks

All experiments use randomly generated Euclidean instances (uniform distribution, minimum inter-city distance $200/N$, map size 600×600). Results averaged over 5 seeds. Instance distributions are verified to be non-trivial: NN-distance coefficient of variation > 0.5 (uneven spacing) with 5–10% of cities in local clusters.

4.1 Tour Quality vs. Optimum (Held-Karp)

For $N \leq 20$, we compute the exact optimum via Held-Karp dynamic programming [7]:

Table 3: Gap above Held-Karp optimum (%), 5 seeds averaged.

N	DSM	NN+2opt	MST+2opt
10	+1.1%	+0.6%	+0.5%
12	+1.5%	+0.2%	+1.5%
15	+1.6%	+0.4%	+3.6%
18	+7.0%	+0.8%	+3.2%
20	+6.0%	+0.8%	+4.5%

DSM tours are 1–7% above optimum. NN+2-opt is consistently closer (<1%). **DSM's tour quality is inferior to NN.** This is the price paid for integer-only arithmetic.

4.2 Tour Quality vs. NN at Scale

Table 4: DSM vs NN+2-opt at larger N (3 seeds).

N	DSM tour	NN tour	Gap
30	2,904	2,781	+4.4%
50	3,535	3,474	+1.7%
100	4,952	4,619	+7.2%

4.3 Matching Precision

Table 5: Antiparallel matching precision with 2 lighthouses. "True" = correctly identified neighbor pairs. "False" = spurious matches.

N	True	False	Precision
50	15	0	100.0%
100	43	0	100.0%
200	135	1	99.3%
300	245	2	99.2%
500	488	11	97.8%

Precision remains above 97% up to N=500 with 2 lighthouses. With 4 lighthouses and tighter tolerance (ϵ=1%), precision at N=500 reaches 99.6%.

4.4 The Real Comparison: MCU Cycle Budget

Tables 3 and 4 show DSM is inferior on tour quality. But tour quality is not DSM's value proposition. The relevant comparison for the target domain is:

Table 6: The comparison that matters: total MCU cycles on an ATmega328 for a complete TSP solution.

N	DSM cycles	NN cycles	Tour gap
100	84K	371K	+7.2%
300	763K	3.36M	+4.4%
1,000	8.5M	37.5M	est. +5%

DSM delivers a TSP tour in 84,000 integer cycles where NN requires 371,000 cycles (including software $\sqrt{\cdot}$). The tour is 7% longer, but it arrives 4.4× faster—and requires no floating-point support of any kind.

5 Related Work

Table 7: DSM in context. "Int-only" means no $\sqrt{\cdot}$, arctan, or FP ops at any stage.

Method	Comp. count	$\sqrt{\cdot}$/edge	Int-only	Guarantee	Quality
Christofides [3]	$O(N^3)$	Yes	No	1.5× OPT	Best
NN+2-opt	$O(N^2)$	Yes	No	None	Good
Sweep [6]	$O(N \log N)$	arctan	No	None	Good
ELG [5]	$O(kN)$	Yes	No (GPU)	None	Good
DSM (ours)	$O(N^2)$	**No**	**Yes**	None	−7%

DSM is the only method in this table that runs entirely on integer hardware. The comparison count is identical to NN; the per-operation cost is lower. Christofides and Sweep are not directly competitive because they require FPU operations that are unavailable on the target hardware. The Sweep algorithm [6] shares the concept of a central reference point (depot ≈ lighthouse) but computes arctan for angular sorting. ELG [5] uses k-NN graphs with polar coordinate features—similar in philosophy to DSM's door model—but requires GPU inference.

6 The Novel Element: Antiparallel Matching

Independent of the TSP application, the *antiparallel compass matching* principle is, to our knowledge, a new method for spatial neighbor discovery:

> *Two points are candidate neighbors if a directional descriptor from each points toward the other (antiparallel) with matching distance.*

This is conceptually distinct from:

- k-NN search (compares distances, ignores direction).
- Delaunay triangulation (geometric construction, requires exact arithmetic).
- Sweep line algorithms (sorts by angle, requires arctan).

The lighthouse filter extends this with a "triangulation by consensus" mechanism: a match is confirmed only if multiple independent reference points agree that the two candidates occupy the same region. This is analogous to GPS positioning (multiple satellites provide redundant location confirmation) but operates on discrete compass sectors rather than continuous coordinates.

We believe this principle may find applications beyond TSP—in ad-hoc network topology discovery, multi-robot rendezvous, or any setting where agents have local directional sensing but no global coordinate system.

7 Limitations

We state these without hedging:

1. Phase 1 is $O(N^2)$. The k-NN search scans all pairs, same as NN. The "linear data" claim applies only to the *output representation* ($10N$ stored values), not to the computation that produces it. Grid-based spatial hashing can reduce this to $O(N\sqrt{N})$ expected case, but worst-case remains quadratic.

2. Tour quality is below NN. DSM tours are 1–7% longer than NN+2-opt, and NN itself is among the weaker TSP heuristics. Against Held-Karp optimum, DSM shows 1–7% gap versus <1% for NN. This gap is intrinsic to the chain-merge construction: unmatched cities require greedy linking that introduces suboptimal connections.

3. No approximation guarantee. Unlike Christofides ($\leq$ 1.5× OPT), DSM provides no worst-case bound. Pathological inputs (e.g., all cities equidistant) could cause complete matching failure.

4. Euclidean 2D only. The compass model requires planar coordinates. It does not apply to road networks, 3D routing, or abstract cost matrices.

5. Modern CPUs see no benefit. On any machine with a hardware FPU (virtually all modern CPUs, including ARM Cortex-M4+), $\sqrt{\cdot}$ costs ∼1 cycle. DSM's advantage vanishes.

8 Target Applications

DSM's value exists in a specific hardware niche. We describe it concretely:

Wireless sensor networks. Hundreds of sensor nodes, each with an 8-bit MCU (ATmega, MSP430), must collectively determine a data-collection tour. Each node knows its coordinates and can compute d^2 and Compass16 to its radio neighbors. The $10N$ door representation can be aggregated over the network with $\sim 100N$ bytes of communication—orders of magnitude less than the $O(N^2)$ full matrix.

Agricultural drone path planning. A low-cost drone with a PIC or AVR MCU must visit N waypoints for crop monitoring. The flight controller has no FPU. With DSM, route computation for N=100 waypoints completes in $\sim$5 ms at 16 MHz—fast enough for real-time re-planning when a waypoint is added or removed.

FPGA routing engines. The Compass16 classifier is a pure combinational circuit: 2 comparators + 2 multipliers + 1 MUX tree, synthesizable in $\sim$200 LUTs on a Xilinx Artix-7. Fully pipelined at 100 MHz, it classifies 10^8 edges per second—enabling real-time TSP heuristics for industrial pick-and-place machines.

9 Conclusion

Directional Sparse Matching is not a better TSP heuristic. It is a *different* one—designed for a hardware regime where established methods cannot run.

The cost is clear: tours are 1–7% longer than Nearest Neighbor. The benefit is equally clear: the entire pipeline runs in integer arithmetic, achieving 4–5× speedup on FPU-less hardware, with a compact $10N$-value graph representation.

The core intellectual contribution is the antiparallel matching principle: neighbor discovery through directional opposition and distance agreement, filtered by lighthouse consensus. We believe this idea—local direction as a proxy for global structure—has value beyond the specific TSP application presented here.

Reproducibility. Python source code for all experiments is available from the author upon request.

References

[1] D. L. Applegate, R. E. Bixby, V. Chvátal, W. J. Cook. *The Traveling Salesman Problem: A Computational Study.* Princeton, 2006.

[2] Microchip Technology. *AVR Instruction Set Manual.* DS40002198, 2021.

[3] N. Christofides. Worst-case analysis of a new heuristic for the TSP. Tech. Rep. 388, CMU, 1976.

[4] G. A. Croes. A method for solving traveling-salesman problems. *Oper. Res.*, 6(6):791–812, 1958.

[5] L. Gao et al. ELG: efficient local-global framework for TSP. *Proc. IJCAI*, 2023.

[6] B. E. Gillett, L. R. Miller. A heuristic for vehicle-dispatch. *Oper. Res.*, 22(2):340–349, 1974.

[7] M. Held, R. M. Karp. A dynamic programming approach to sequencing problems. *J. SIAM*, 10(1):196–210, 1962.

[8] R. M. Karp. Reducibility among combinatorial problems. In *Complexity of Computer Computations*, pp. 85–103. Plenum, 1972.

[9] S. Lin, B. W. Kernighan. An effective heuristic for TSP. *Oper. Res.*, 21(2):498–516, 1973.

[10] A. I. Serdyukov. On some extremal walks in graphs. *Upravlyaemye Sistemy*, 17:76–79, 1978.